

Project Statement: Multi-Card ATM Simulator

1. Problem Statement

Traditional simplified ATM terminal simulations often support only a single hardcoded account or fixed card credentials. This approach fails to reflect real-world banking environments where an ATM system must manage multiple user accounts, enforce access controls, maintain isolated transactional histories, and persist state across runtime sessions.

The **Multi-Card ATM Simulator** addresses these limitations by offering an object-oriented Python application capable of managing multiple bank cards dynamically with secure access controls and persistent state management.

2. Project Objectives

The objective of this project is to develop a robust, menu-driven CLI (Command Line Interface) application in Python that simulates multi-account ATM interactions.

Key objectives include:

- **Account Management:** Enable creation, listing, selection, and deletion of multiple card accounts.
- **Security & Authentication:** Restrict access to individual card operations using PIN verification with retry limits and account masking.
- **Core ATM Operations:** Allow deposit, withdrawal, balance inquiry, and PIN modification functions.
- **Audit Logging:** Maintain a transaction history per card and expose a mini-statement feature (displaying recent activity).
- **Data Persistence:** Store and retrieve all card profiles and transaction logs to/from a local JSON database file.

3. Scope of the System

In Scope

1. **Dynamic Account Creation:** Generating unique 16-digit card numbers with custom initial deposits and user-defined 4-digit PINs.
2. **Flexible Account Retrieval:** Searching/selecting accounts via either full 16-digit card numbers or the last 4 digits.
3. **Session Management:** Secure login sessions requiring PIN validation, supporting account switching and clean logging out.
4. **Transaction Integrity:** Ensuring non-negative input validation for deposits/withdrawals and

preventing overdrafts on insufficient funds.

5. **Persistence Layer:** Real-time data serialization using JSON (atm_cards.json).

Out of Scope (Future Enhancements)

- GUI (Graphical User Interface) or Web Interface.
- Integration with real banking networks or payment gateways.
- Encryption algorithms for stored PINs (currently stored in plain JSON text for simulation simplicity).
- Hardware interface integration (e.g., physical card readers or cash dispensers).

4. Functional Requirements

4.1 Card Management

- **Create Card:** Generates a unique 16-digit card number and attaches holder name, 4-digit numeric PIN, and opening balance.
- **List Cards:** Displays cardholder names, masked card numbers (**** * 1234), and current balances.
- **Delete Card:** Removes a card profile from system state upon user confirmation.

4.2 Security & Authentication

- Maximum 3 consecutive failed PIN attempts allowed before returning to the main menu.
- Masking sensitive card numbers across displays.
- PIN change operation requires validation of the existing PIN and strict format compliance (4 numeric digits).

4.3 Banking Operations

- **Balance Check:** Displays current account balance in standard currency formatting.
- **Deposit:** Validates positive input values, adds funds to current balance, and logs the transaction.
- **Withdrawal:** Checks for sufficient funds and positive input values before deducting balance and logging the transaction.
- **Mini Statement:** Shows up to the last 5 transactions formatted with timestamps (YYYY-MM-DD HH:MM:SS).

5. Technical Architecture

- **Language:** Python 3.x
- **Core Libraries Used:**
 - json (Data persistence)
 - os (File existence validation)
 - random (16-digit card number generation)
 - datetime (Timestamping transactions)
- **Design Pattern:** Object-Oriented Programming (OOP) featuring a Card domain class and

an ATMSystem manager class.

6. Target Audience & Practical Utility

This simulator serves as an educational tool for demonstrating software engineering fundamentals, including:

- Object-Oriented System Design.
- Data Serialization and Deserialization patterns.
- CLI User Experience & Input Validation defensive programming techniques.
- State and Session Management in modular Python applications.

7. Author

Name: Tanmay Shrivas

Registration Number : 26BCE10603